

Structures & Unions

Structures

- **We've** seen variables of simple data types, such as float, char, and int.
- Variables of such types represent one item of information: a height, an amount, a count, and so on.
- But just as groceries are organized into bags, employees into departments, and words into sentences, it's often convenient to organize simple variables into more complex entities.
- The C++ construction called the *structure* is one way to do this.

Structures

- A structure is a collection of one or more variables, possibly of different types, grouped together under a single name for convenient handling .
- The variables in a structure can be int, some can be float, and so on. This is unlike the array, in which all the variables must be the same type. The data items in a structure are called the **members** of the structure.

- **Structures assist program organisation by**
 - Grouping logically related data, and giving this set of variables a higher-level name and more abstract representation.
 - Enabling related variables to be manipulated as a single unit rather than as separate entities.
 - Reducing the number of parameters that need to be passed between functions.
 - Providing another means to return multiple values from a function.

- The general format of a structure definition is

```
struct tag_name
{
    Data_type member1;
    Data_type member2;
    -----
};
```

Example:-1

```
struct student
{
    int rollno;
    int age;
    char name[10];
    float height;
};
```

```
struct Student
{
    int rollno;
    int age;
    char name[20];
    char branch[20];
    double avg_marks;
};
```

Student S;



Address

2000		S.rollno	(4 bytes)	
2004		S.age	(4 bytes)	
2008		S.name[20]	(20 bytes)	
2028		S.branch[20]	(20 bytes)	
2048		S.avg_marks	(8 bytes)	
2056				

Declaring Structure Variables

Example-2

1. Declare them at the structure definition.

```
struct student
{
    int rollno;
    int age;
    char name[10];
    float height;
}s1, s2, s3;
```

2. Define the variables at some point *after* the structure definition.

```
student s1, s2, s3;
```

Note:

- Members of a structure themselves are not variables.
- They (members) do not occupy any memory until they are associated with the structure variables such as s1, s2, s3 etc.
- Normally, structure definitions appear at the beginning of the program file, before any variables or functions are defined.
- They may also appear before main. In such cases, the definition is global and can be used by other functions as well.

- The **sizeof** operator is used to find the size of structure.

sizeof(struct x) → gives the number of bytes required to hold all the members of the structure x.

- If y is a simple structure variable of type **struct x**, then the expression **sizeof(y)** → would also give the same answer.

Here is how we would assign values to the members of **student s1**

```
strcpy(s1.name, "Anil");
```

```
s1.rollno=1335;
```

```
s1.age=18;
```

```
s1.height=5.8;
```

We can also use `cin` to give the values through the keyboard.

```
cin>>s1.name>>s1.age>>s1.rollno>>s1.height;
```

Example-3

```
4 struct personal
5 {
6     char name[20];
7     int day;
8     char month[10];
9     int year;
10    float salary;
11 };
```

```
12 int main()
13 {
14     personal person;
15     cout<<"Input values\n";
16     cin>>person.name>>person.day
17         >>person.month>>person.year
18         >>person.salary;
19
20     cout<< person.name<<endl
21         <<person.day<<endl
22         <<person.month<<endl
23         <<person.year<<endl
24         <<person.salary<<endl;
25 }
```

Structure initialization

Example-4

```
struct part
{
    int  modelnumber;
    int  partnumber;
    float cost;
};
int main()
{
    part part1 = { 6244, 373, 217.55 };
}
```

Comparison of structure variables

- *If person1 and person2 belong to the same structure, then the following operation is valid.*

person1 = person2; // Assign person2 to person1

But if(person1==person2) // illegal

if(person1!=person2) //illegal

Example-5

```
4 struct person
5 {
6     char name[20];
7     int age;
8     float salary;
9 };
10 int main()
11 {
12     person person1 = {"abc", 28, 36000};
13     person person2 = {"xyz", 25, 30000 }, p3;
14
15     p3 = person2;
16
17     cout<<p3.name<<"\n"<<p3.age<<"\n"<<p3.salary<<"\n";
18
19     p3 = person1;
20
21     cout<<p3.name<<"\n"<<p3.age<<"\n"<<p3.salary;
22     return 0;
23 }
```

Arrays of structures

For example,

`class1 student[100];` → *defines an array called student, that consists of 100 elements.*

Example-6

```
3 struct marks
4 {
5     int subject1, subject2, subject3;
6 };
7 int main()
8 {
9     marks students[] = { {45, 67, 78}, {75, 55, 69}, {55, 79, 90} };
10    // .....
11 }
```

*This declares the **student** as an array of 3 elements and initializes their members as follows.*

student[0].subject1=45;

student[0].subject2=67;

student[0].subject3=78;

.....

.....

student[2].subject3=90;

```
marks students[] = { 45, 67, 78, 75, 55, 69, 55, 79, 90 };
```


Arrays within structures

We can use single or multidimensional arrays inside a structure.

```
struct marks  
{  
  int   number;  
  float sub[3];  
} student[2];
```

The member sub contains 3 elements sub[0], sub[1],sub[2]. These elements can be accessed using appropriate subscripts.

```

4 struct marks
5 {
6     int sub[3];
7     int total;
8 };
9
10 int main()
11 {
12     marks student[3] = { {45,67,78,0}, {75,55,69,0}, {55,79,90,0} };
13     marks subtotal{0,0,0,0};
14     // ...
15 }

```

`student[1].sub[2]` → *refers to the mark in third subject by the second student.*

STRUCTURES WITHIN STRUCTURES

```
3 struct date
4 {
5     int day, month, year;
6 };
7
8 struct student
9 {
10     unsigned int RegNo;
11     char name[20];
12     date birth_date;
13 };
14
15 int main()
16 {
17     student s = {111, "Anil", 20, 5, 2000 };
18     cout<<s.RegNo<<"\n"<<s.name<<"\n"
19     <<s.birth_date.day<<"-"<<s.birth_date.month
20     <<"-"<<s.birth_date.year;
21     // .....
22     return 0;
23 }
```

```

4  struct student
5  {
6      unsigned int RegNo;
7      char name[20];
8      struct date
9      {
10         int day, month, year;
11     } birth_date;
12 };
13
14 int main()
15 {
16     student s = {111, "Anil", 20, 5, 2000 };
17     cout<<s.RegNo<<"\n"<<s.name<<"\n"
18     <<s.birth_date.day<<"-"<<s.birth_date.month
19     <<"-"<<s.birth_date.year;
20     // .....
21     return 0;
22 }

```

- *It is also permissible to nest more than one type of structures.*

```
struct personal_record
{
    struct name_part name;
    struct addr_part address;
    struct date date_of_birth;
    .....
    ....
};
struct personal_record person1;
```

Unions

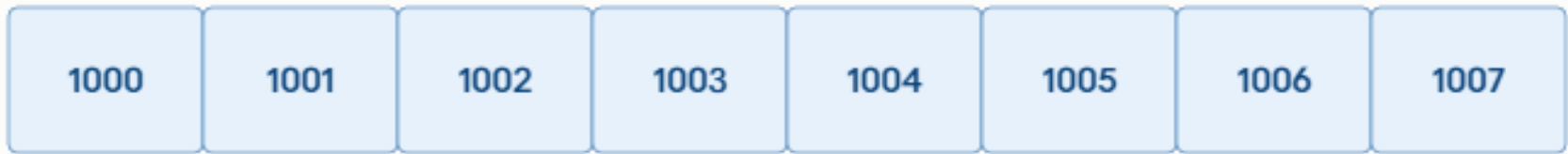
- ✓ *Unions look similar to structures.*
- ✓ *They have identical declaration syntax and member access.*
- ✓ *The major difference between them in terms of storage.*
- ✓ *In structures, each member has its own storage location.*
- ✓ *Whereas all the members of a union use the same location.*
- ✓ *This implies that, although a union may contain many numbers of different types. It can handle only one member at a time.*

Example:

```
3 union item
4 {
5     int i;
6     double d;
7     char c;
8 };
```

```
10 int main()
11 {
12     item I;
13
14     I.i = 11;
15     cout << I.i << "\n";
16     // cout << I.d << "\n";    // Invalid
17     // cout << I.c << "\n";    // Invalid
18
19     I.d = 22.5;
20     cout << I.d << "\n";
21     // cout << I.i << "\n";    // Invalid
22     // cout << I.c << "\n";    // Invalid
23
24     I.c = 'a';
25     cout << I.c;
26     // cout << I.i << "\n";    // Invalid
27     // cout << I.d << "\n";    // Invalid
28
29     return 0;
30 }
```

- *A union holds the value of one-variable at a time.*
- *The compiler allocates a piece of storage that is large enough to hold the biggest member of the union.*
- *In the declaration below, the member 'd' requires 8 bytes which is the largest among the members, All members (i, d, c) then share that same 8-byte block starting at address 1000*



c  1 byte (char)

i  4 bytes (int)

d  8 bytes (double)